

EduSync 全栈开发学习手册

主要使用 Cursor / AI 写代码、希望系统补课的 EduSync 作者（Zachary）

版本：2026年6月 · 第二版（全栈 + AI + 部署）

网站：<https://edu-sync-gamma.vercel.app>

适用对象：主要使用 Cursor / AI 写代码、希望系统补课的 EduSync 作者（Zachary）

目录

1. 你为什么需要这份手册
2. 你的项目全景（真实环境）
3. 全栈架构：一张图看懂
4. 数据是如何传输的（完整链路）
5. 前端知识体系（结合你的代码）
6. 后端知识体系（Flask + Python）
7. Supabase 深度解析
8. 认证与登录完整流程
9. AI 功能架构（DeepSeek）
10. 部署：Vercel + Railway + Git
11. 环境变量完整字典
12. 你用 Cursor 开发时漏掉的知识点
13. 你项目里踩过的坑 → 对应知识点
14. 代码库地图（按目录）
15. 日常开发命令速查
16. 术语表（中英对照）
17. 延伸阅读与官方文档
18. 总结：你现在该记住的十句话

从 Cursor 辅助开发到真正理解你的项目

版本： 2026年6月 · 第二版（全栈 + AI + 部署）
网站： <https://edu-sync-gamma.vercel.app>
适用对象： 主要使用 Cursor / AI 写代码、希望系统补课的 EduSync 作者（Zachary）

本手册结合 你当前真实线上环境、仓库里实际代码、以及 开发过程中踩过的坑 编写。
第一版 OAuth 专题见： [docs/EduSync-Google-OAuth-Deploy-Learning-Guide.pdf](#)（本册会涵盖并扩展）。

1. 你为什么需要这份手册

1.1 Cursor 开发 vs 真正学会

你用 Cursor 时	你可能没真正掌握
「帮我加一个登录页」	React 组件、状态、路由如何配合
「部署到 Vercel」	构建时注入 vs 运行时、CORS、环境变量
「接 Supabase」	Auth、PostgreSQL、RLS、service_role 区别
「修这个 500 错误」	请求从浏览器到数据库的完整链路

结论： Cursor 是加速器，不是替代品。你要能回答三个问题：

1. 数据从哪来、到哪去？（浏览器 → API → 数据库 → 返回）
2. 谁负责哪一层？（Vercel / Railway / Supabase / Google 各干什么）
3. 改一个地方会影响哪里？（例如改 `VITE_API_URL` 必须重新 build）

1.2 本手册覆盖范围

- 全栈架构与数据流（含图示）
- 前端：React、Vite、TypeScript、Tailwind、React Query、路由守卫
- 后端：Flask、Blueprint、中间件、Supabase Python 客户端
- 数据库：Supabase PostgreSQL、表关系、SQL 迁移文件
- 认证：邮箱密码、Google OAuth、JWT、localStorage
- AI：DeepSeek、SSE 流式、工具调用（read tools）
- 部署：Vercel（前端）+ Railway（后端）+ 环境变量大全
- 你用 Cursor 时容易漏掉的知识点 + 学习路线

2. 你的项目全景（真实环境）

2.1 线上地址一览

组件	平台	地址 / 说明
前端（用户访问）	Vercel	<code>https://edu-sync-gamma.vercel.app</code>
后端 API	Railway	<code>https://edusync-production-6d33.up.railway.app/api</code>
数据库 + Auth	Supabase	<code>https://ptxrmujnqrwakfpdyhh.supabase.co</code>
AI 模型	DeepSeek	仅后端调用，密钥在 Railway Variables
代码仓库	GitHub	<code>fanxiaotuGod/EduSync</code>
邮件（可选）	Resend	后端 <code>RESEND_API_KEY</code>

2.2 EduSync 是什么（产品层）

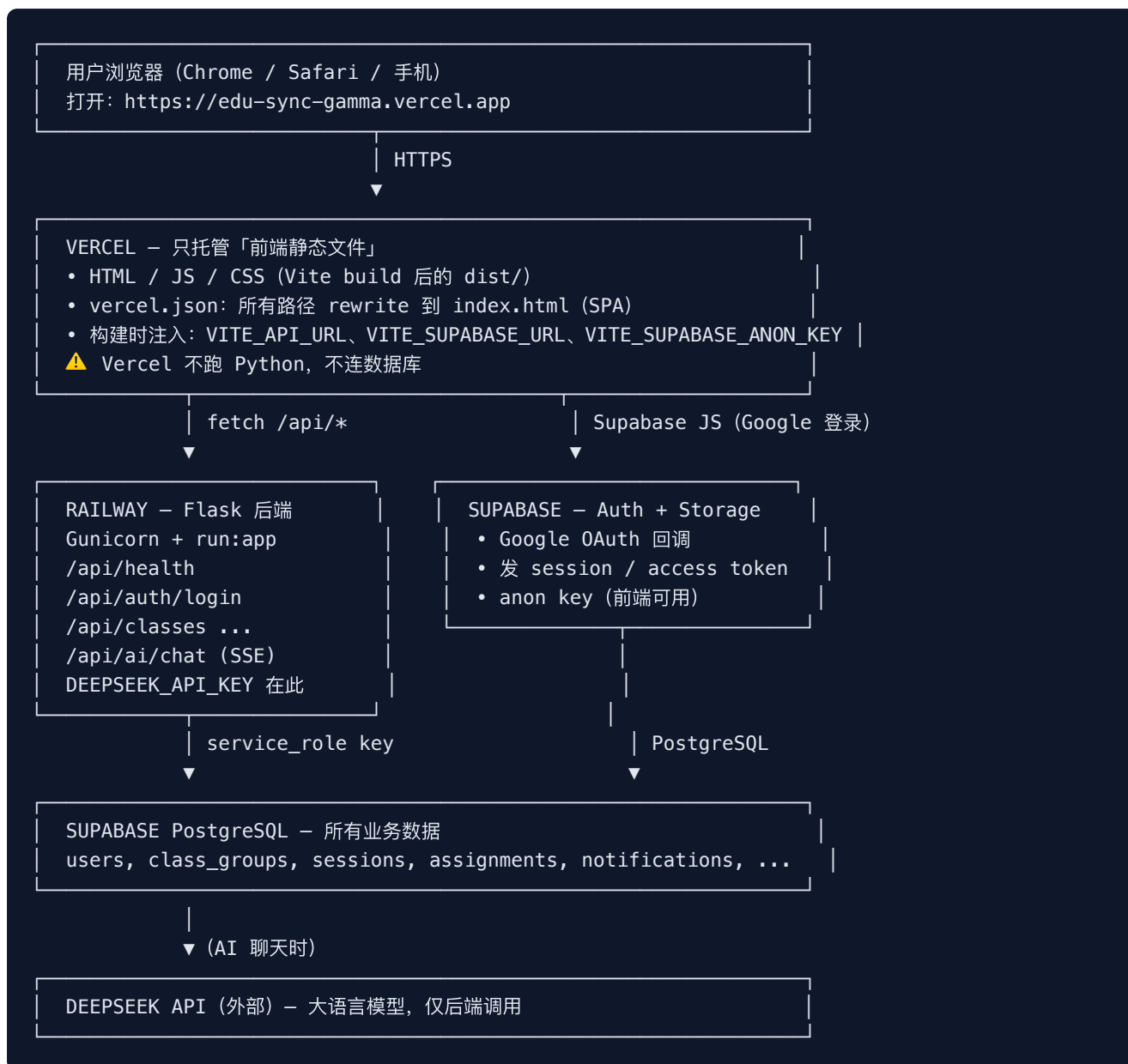
面向 **老师 + 学生** 的教学协作平台：

- 班级与邀请码、学生账号
- 共享日历 / 排课、出勤、调课申请
- 作业发布与提交
- 学费 / 课时余额
- 通知
- **老师端 AI 助手**（Beta，只读查询 + 对话）
- **新功能预告投票**（拖拽排课、学生资料 AI、微信/WhatsApp、周期报告、Google Drive 等）

2.3 技术栈总表

层级	技术	在你项目里的角色
UI	React 18 + TypeScript	所有页面与组件
构建	Vite	开发服务器、 <code>npm run build</code> 产出 <code>dist/</code>
样式	Tailwind CSS + shadcn/ui	按钮、对话框、侧边栏等
数据请求	TanStack React Query	缓存 API 结果、自动 refetch
国际化	i18next	<code>src/locales/zh.json</code> 、 <code>en.json</code>
路由	React Router v6	<code>App.tsx</code> 里定义路径
API 封装	<code>src/lib/api.ts</code>	<code>apiFetch</code> 、各业务函数
后端框架	Flask (Python)	<code>backend/app/</code>
ORM/DB 访问	Supabase Python SDK	直接 <code>.table().select()</code>
数据库	PostgreSQL (Supabase)	所有业务表
身份认证	Supabase Auth + 自签 JWT	登录、Google OAuth
文件存储	Supabase Storage	班级资料、作业文件
前端托管	Vercel	静态站点 + SPA 路由
后端托管	Railway	Gunicorn 跑 Flask
LLM	DeepSeek API	<code>/api/ai/chat</code> 流式回复

3. 全栈架构：一张图看懂



3.1 关键理解（必背）

1. 前端和后端是两次部署、两个域名 — 浏览器用 `fetch` 跨域访问 Railway，靠 **CORS** 允许。
2. Supabase 有两套密钥 — `anon`（前端）和 `service_role`（仅后端，权限极大，绝不能进前端或 Git）。
3. Vite 环境变量以 `VITE_` 开头 — 只在 `npm run build` 时 打进 JS；改 Vercel 变量后必须 **Redeploy**。
4. Railway 环境变量在进程启动时读取 — 改完通常自动重新部署；不需要「build 前端」那种概念。

4. 数据是如何传输的（完整链路）

4.1 例子：老师打开日历页，看到课程列表

```
1. 用户已登录 → localStorage 有 edusync_token
2. CalendarPage 挂载 → useQuery 调用 listSessions()
3. listSessions() → apiFetch('/sessions?month=2026-06')
4. apiFetch 拼接 BASE_URL:
   生产: https://edusync-production-6d33.up.railway.app/api/sessions?month=...
   本地: http://127.0.0.1:5000/api/sessions?month=...
5. 请求头自动加: Authorization: Bearer <JWT>
6. Railway 上 Flask → sessions blueprint → @require_auth
7. require_auth 用 supabase_auth.auth.get_user(token) 验证 JWT
8. 查 PostgreSQL: sessions 表 + class_groups 权限
9. 返回 JSON 数组
10. React Query 缓存数据 → 页面渲染列表
```

4.2 HTTP 请求里有什么

部分	示例	谁关心
Method	GET / POST / PATCH / DELETE	REST 语义
URL	/api/sessions	路由到哪个 Blueprint
Headers	Authorization: Bearer eyJ...	后端鉴权
Headers	Content-Type: application/json	POST body 格式
Body	{"class_id": "...", "date": "2026-06-15"}	创建/更新数据
Response	200 + JSON 或 401 / 403 / 500	前端 toast / 跳转登录

4.3 前端 api.ts 的设计（你应该会读）

文件: src/lib/api.ts

- BASE_URL = import.meta.env.VITE_API_URL 或本地默认 http://127.0.0.1:5000/api
- apiFetch(path, init): 自动加 Bearer token (从 localStorage 的 edusync_token)
- 每个业务一个函数: listClasses()、createSession()、streamAiChat() 等

学习作业：打开 CalendarPage.tsx，搜 listSessions，跟到 api.ts，再跟到 backend/app/blueprints/sessions.py（或类似文件）。

4.4 后端 Blueprint 模式

文件: backend/app/__init__.py 注册所有 blueprint:

Blueprint	大致职责
health_bp	GET /api/health 健康检查
auth_bp	注册、登录、OAuth
users_bp	个人资料
classes_bp	班级 CRUD、资料上传
sessions_bp	日历课程
students_bp	学生管理、邀请
assignments_bp	作业
tuition_bp	学费余额
notifications_bp	通知
ai_bp	AI 聊天 SSE
feedback_bp	新功能投票

每个 blueprint 是一个 Python 文件，用 `@blueprint.route` 定义 URL。

4.5 中间件: `require_auth` 与 `require_role`

文件: `backend/app/middleware/auth.py`

```
require_auth:
    读 Authorization 头 → 取 Bearer token → supabase_auth.get_user(token)
    → 成功则 g.current_user = 用户 → 继续执行视图函数
    → 失败返回 401

require_role('teacher'):
    在 require_auth 之后 → 查 users 表 role → 不是 teacher 则 403
```

5. 前端知识体系（结合你的代码）

5.1 React 核心概念

概念	在你项目中的例子
组件	<code>CalendarPage.tsx</code> 、 <code>AiAssistant.tsx</code>
Props	父传子，如 <code><AiAssistant variant="embedded" /></code>
State	<code>useState</code> ： 表单、对话框开关
Effect	<code>useEffect</code> ： 页面加载时同步 URL 参数
Context	<code>AuthContext</code> ： 全局 user / token / login / logout
自定义 Hook	<code>useAuth()</code> 、 <code>use-mobile.tsx</code>

5.2 React Router（路由）

文件： `src/App.tsx`

路径	组件	守卫
<code>/</code>	<code>HomePage</code>	无（已登录则跳 dashboard）
<code>/login</code>	<code>LoginPage</code>	<code>GuestRoute</code> （已登录不能进）
<code>/register</code>	<code>RegisterPage</code>	<code>GuestRoute</code>
<code>/auth/callback</code>	<code>AuthCallbackPage</code>	Google OAuth 回调
<code>/dashboard</code> 等	各业务页	<code>ProtectedRoute</code> （未登录 → <code>/login</code> ）

`ProtectedRoute`（ `src/components/ProtectedRoute.tsx` ）： 没有 token → `<Navigate to="/login" />`

`GuestRoute`： 已有 token → 跳 dashboard

5.3 TanStack React Query

为什么用：避免每个页面自己 `useEffect + fetch`，统一 loading/error/缓存。

```
const classesQuery = useQuery({
  queryKey: ["classes", user?.id],
  queryFn: () => listClasses(),
  enabled: Boolean(user?.id),
});
```

- `queryKey`：缓存键，数据变了用 `queryClient.invalidateQueries`
- `enabled`：没登录就不请求

5.4 Vite 是什么

- 开发: `npm run dev` → 本地 `http://localhost:8080`, 热更新
- 生产: `npm run build` → 输出 `dist/` 文件夹 (纯静态)
- 环境变量: 只有 `VITE_` 前缀会暴露给前端代码

5.5 Tailwind + shadcn/ui

- Tailwind: 用 class 写样式, 如 `flex gap-2 rounded-xl`
- shadcn: 可复制组件在 `src/components/ui/`, 基于 Radix UI

5.6 国际化 i18n

- 文案在 `src/locales/zh.json`、`en.json`
- 页面里: `const { t } = useTranslation();` → `t("calendar.title")`
- 改 UI 文字优先改 json, 不要硬编码英文 (HomePage 营销页除外)

6. 后端知识体系 (Flask + Python)

6.1 项目入口

```
backend/  
  run.py          → from app import create_app; app = create_app()  
  Procfile        → gunicorn run:app --bind 0.0.0.0:$PORT (Railway 用)  
  app/  
    __init__.py   → create_app() 注册 blueprint  
    config.py      → 从环境变量读配置  
    extensions.py  → supabase 客户端单例  
    blueprints/    → 各 API 模块  
    middleware/    → auth 装饰器  
    services/      → 业务逻辑 (email、AI、学生账号等)
```

6.2 Config 与 extensions

- `Config` 类: 启动时检查 `SUPABASE_URL`、`SUPABASE_SERVICE_ROLE_KEY` 等是否缺失
- `extensions.py`: `supabase` 用 `service_role` 连接, 可绕过 RLS 做后端操作

6.3 本地运行后端

```
cd backend
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env      # 填 Supabase 等
python run.py             # 默认 http://127.0.0.1:5000
curl http://127.0.0.1:5000/api/health
```

6.4 Gunicorn (生产)

Railway 不用 `python run.py`，用 **Gunicorn** 多 worker 跑 WSGI 应用，更稳。

7. Supabase 深度解析

7.1 Supabase 不是「只有一个数据库」

它是一套 BaaS (Backend-as-a-Service)：

模块	作用
PostgreSQL	存 users、sessions、classes...
Auth	注册、登录、OAuth、发 JWT
Storage	上传 PDF、作业文件 (bucket)
Dashboard SQL Editor	跑 <code>backend/sql/*.sql</code> 迁移

7.2 两把钥匙 (最重要)

密钥	在哪用	权限
anon (public)	前端 <code>VITE_SUPABASE_ANON_KEY</code>	受 RLS 限制，适合客户端
service_role	后端 <code>SUPABASE_SERVICE_ROLE_KEY</code>	几乎全权，绝不能泄露

你的 Flask 后端用 `service_role` 直接操作表；前端 Google 登录用 `anon` + Supabase JS SDK。

7.3 主要数据表（逻辑关系）

```
users (用户: 老师/学生)
├── class_groups (班级, teacher_id 指向 users)
│   ├── class_enrollments (学生加入班级)
│   ├── sessions (排课)
│   ├── assignments (作业)
│   └── class_materials (资料文件)
├── notifications (通知)
├── student_balances / transactions (学费)
├── attendance (出勤)
├── feature_feedback (新功能投票)
└── ai_interactions (AI 对话日志, 可选)
```

7.4 SQL 迁移文件在哪

目录: `backend/sql/`

常见:

- `create_mvp_tables.sql` — 基础表
- `fix_sessions_schema.sql`、`fix_class_groups_schema.sql` — 修列
- `create_feature_feedback.sql` — 功能投票
- `create_ai_interactions.sql` — AI 日志
- `setup_materials_bucket.sql` — Storage 桶

操作: Supabase 控制台 → SQL Editor → 粘贴 → Run (每个文件通常只需跑一次)。

7.5 RLS (Row Level Security)

Supabase 表可开 RLS 限制「谁能读哪一行」。

你的架构是 后端用 `service_role` 统一访问, 前端 不直接查表 (除 OAuth 用 Auth API)。这是常见 BFF (Backend For Frontend) 模式。

8. 认证与登录完整流程

8.1 邮箱 + 密码登录

1. LoginPage 提交 email/password
2. POST `/api/auth/login` → auth blueprint
3. 后端查 users、校验密码 (Supabase Auth 或 bcrypt, 以实现为准)
4. 返回 `{ token, user: { id, display_name, role } }`
5. `AuthContext.login(token, user)` → 写入 `localStorage` + React state
6. navigate 到 `/dashboard` 或 `from` 参数指向的页面

localStorage 键:

- `edusync_token` — JWT
- `edusync_user` — JSON 用户信息

8.2 Google 一键登录 (OAuth)

详见第一版 PDF; 核心两步跳转:

1. 用户点 Google → Supabase → **Google** 授权 → 回到 `https://xxx.supabase.co/auth/v1/callback`
2. Supabase → 重定向到你的 `/auth/callback`, 带 token 或 code
3. `AuthCallbackPage` 调后端 `POST /api/auth/oauth/complete` 或 `register`
4. 后端在 `users` 表建/查记录, 返回你自己的 JWT

配置位置:

- Google Cloud Console: redirect URI = Supabase callback
- Supabase: Redirect URLs = 你的 Vercel 域名 `/auth/callback`

8.3 老师添加学生账号

- 后端 `student_accounts.py`: 创建 Supabase Auth 用户、默认密码 `123456`、确认邮箱
- 学生用 登录 而非注册; 常见问题「Email not confirmed」已通过服务端 `confirm` 修复

8.4 每次 API 请求如何带身份

`apiFetch` 自动: `Authorization: Bearer <edusync_token>`

后端 `@require_auth` 验证 → 业务逻辑里用 `g.current_user.id`

9. AI 功能架构 (DeepSeek)

9.1 设计原则

- 密钥只在后端 — `DEEPSEEK_API_KEY` 在 `backend/.env` / Railway, Never `VITE_*`
- 仅老师可用 — `@require_role('teacher')` on `ai_bp`
- Beta 阶段只读 — AI 可调 `read tools` 查班级/课表, 写操作需未来 `confirm` 流程

9.2 数据流：老师发一条 AI 消息

```
1. AiAssistant 组件 → streamAiChat(messages)
2. POST /api/ai/chat, body: { messages: [...] }
3. Flask 调 DeepSeek API (backend/app/services/deepseek.py)
4. 若模型要查数据 → ai_tools.py 读 Supabase
5. 响应用 SSE (Server-Sent Events) 流式返回 token
6. 前端 ReadableStream 逐字显示
7. 可选写入 ai_interactions 表
```

9.3 SSE 事件类型

type	含义
token	一段文字
tool_start	开始查数据库
tool_done	查完
done	结束
error	出错

9.4 相关文件

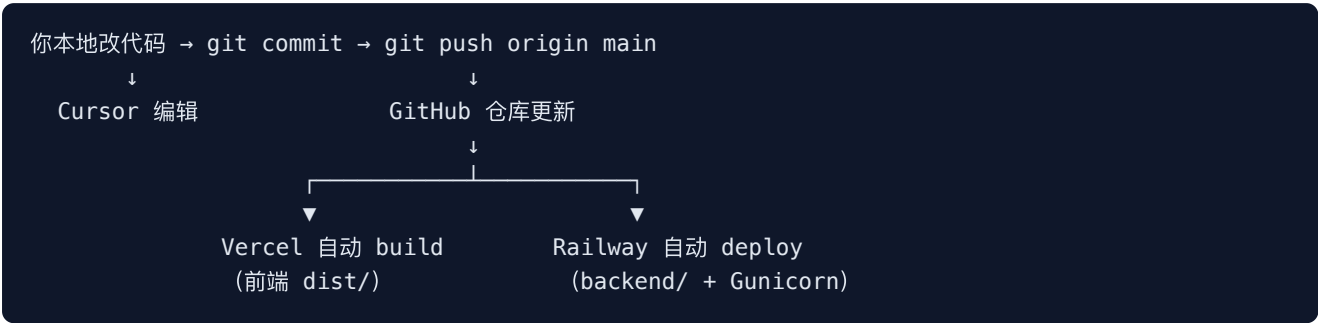
- backend/app/blueprints/ai.py
- backend/app/services/deepseek.py 、 ai_tools.py
- src/components/AiAssistant.tsx
- docs/AI-ARCHITECTURE.md 、 docs/AI-SAFETY-POLICY.md

9.5 路线图上的 AI 功能（预告投票）

feature_id	说明
calendar_drag_schedule	拖拽排课
student_materials_ai	学生基于老师资料问答
messaging_ai_integration	微信/WhatsApp 连 AI
periodic_teacher_reports	周期学情报告
google_drive_integration	Google Drive 资料接入

10. 部署：Vercel + Railway + Git

10.1 Git 与 GitHub 在流程中的位置



10.2 Vercel 详解

它是什么：静态网站 + Serverless 边缘 CDN 托管商。

EduSync 配置：

- 根目录：仓库根（不是 `backend/`）
- Build: `npm run build`
- Output: `dist`
- `vercel.json`: `rewrites` 让所有路径回到 `index.html` (React SPA 必须，否则刷新 `/calendar` 会 404)

环境变量 (Production)：

变量	值
<code>VITE_API_URL</code>	<code>https://edusync-production-6d33.up.railway.app/api</code>
<code>VITE_SUPABASE_URL</code>	你的 Supabase URL
<code>VITE_SUPABASE_ANON_KEY</code>	anon key

⚠ 改完必须 **Redeploy**，否则旧 JS 里仍是旧 API 地址。

10.3 Railway 详解

它是什么：跑长期在线进程的平台（你的 Flask）。

关键设置：

- Root Directory: `backend`
- Start: `gunicorn run:app --bind 0.0.0.0:$PORT` (见 `Procfile`)
- Variables: 见下一节

试用限制：Trial 约 30 天或 \$5 额度用完；长期需 Hobby 付费。

健康检查： GET /api/health → {"status":"ok"}

10.4 只部署 Vercel 可以吗？

不行。 前端静态页没有你的业务 API；必须 Railway（或 Render、自建 VPS）同时在线。

10.5 CORS

backend/app/__init__.py 里 CORS(app) 允许浏览器从 vercel.app 域名访问 Railway API。
FRONTEND_URL 环境变量用于邮件链接、OAuth 等需要拼前端地址的地方。

11. 环境变量完整字典

11.1 前端（.env 或 Vercel）

变量	必填	说明
VITE_API_URL	生产必填	后端根，含 /api
VITE_SUPABASE_URL	Google 登录必填	Supabase 项目 URL
VITE_SUPABASE_ANON_KEY	Google 登录必填	公开 anon key

11.2 后端（backend/.env 或 Railway）

变量	必填	说明
SUPABASE_URL	✔	同上前缀
SUPABASE_SERVICE_ROLE_KEY	✔	service_role，保密
FRONTEND_URL	推荐	https://edu-sync-gamma.vercel.app
FLASK_ENV	推荐	production
DEEPSEEK_API_KEY	AI 必填	DeepSeek
DEEPSEEK_API_BASE	可选	默认 deepseek.com
DEEPSEEK_MODEL	可选	默认 deepseek-chat
RESEND_API_KEY	邮件功能	Resend
CRON_SECRET	定时任务	调 /api/cron/*
DEFAULT_STUDENT_PASSWORD	可选	默认 123456

11.3 绝对不要提交 Git 的

- 任何 `.env`
- `SUPABASE_SERVICE_ROLE_KEY`
- `DEEPSEEK_API_KEY`
- Google Client Secret

`.gitignore` 应已忽略；用 `.env.example` 做模板。

12. 你用 Cursor 开发时漏掉的知识点

12.1 建议补课顺序（4—8 周）

周	主题	具体行动
1	HTTP + REST	用浏览器 F12 Network 跟一次登录、一次 listSessions
2	React 基础	读 <code>AuthContext</code> 、 <code>ProtectedRoute</code> 、一个完整 Page
3	Flask 路由	读 <code>auth.py</code> 、 <code>classes.py</code> 各一个 endpoint
4	Supabase + SQL	在 Dashboard 看表结构，跑一条 SELECT
5	部署链路	自己改一行文案 → push → 看 Vercel 部署日志
6	AI/SSE	读 <code>AiAssistant</code> + <code>ai.py</code> 流式部分
7	安全	anon vs service_role、JWT、CORS
8	综合	独立加一个小 API（如 GET /api/ping-you）不用 Cursor

12.2 每个平台「一句话」

名词	一句话
Vercel	托管你 build 出来的前端网页
Railway	24 小时运行你的 Python 后端
Supabase	云上的 PostgreSQL + 登录 + 文件桶
GitHub	代码版本库，触发自动部署
Vite	把 React 源码打包成浏览器能跑的 JS
Flask	Python 里写 URL → 函数的轻量 Web 框架
JWT	登录后发的「通行证」，每次 API 带上
OAuth	用 Google 登录而不把 Google 密码给你
SSE	服务器一边生成 AI 字一边推给浏览器
CORS	浏览器允许 A 网站请求 B 网站 API 的规则
Blueprint	Flask 里按模块拆分路由的方式
React Query	管 API 数据缓存和刷新的库

12.3 Cursor 使用建议

- 1. 让 AI 改代码后，你自己读 diff 一问「这段为什么这样写」
 - 2. 大功能先画数据流 — 再让 Cursor 实现
 - 3. 部署问题先查日志 — Vercel Build Log、Railway Deploy Log
 - 4. 数据库变更自己跑 SQL — 知道执行了哪条 migration
-

13. 你项目里踩过的坑 → 对应知识点

现象	根因	学到什么
学生无法登录 123456	Supabase Email not confirmed	Auth 状态机、服务端 confirm
Google 登录 redirect 错误	两套 redirect URL 混了	OAuth 流程、Supabase vs Google 配置
Failed to fetch	VITE_API_URL 错或未 redeploy	构建时环境变量
Railway ModuleNotFoundError	Root Directory 不是 backend	部署目录、Procfile
column does not exist	SQL 迁移未跑	Schema 版本与 backend/sql/
AI 双滚动条	嵌套 overflow	CSS flex + overflow 布局
功能投票 503	feature_feedback 表不存在	先跑 SQL 再调 API

14. 代码库地图（按目录）

```
EduSync/
├── src/                                # 前端源码
│   ├── App.tsx                        # 路由总表
│   ├── pages/                         # 每个 URL 一个页面
│   ├── components/                   # 可复用 UI (含 ui/ shadcn)
│   ├── context/AuthContext.tsx      # 登录状态中枢
│   ├── lib/api.ts                    # 所有 HTTP API
│   ├── lib/roles.ts                 # teacher/student 判断
│   └── locales/                      # 中英文文案
├── backend/
│   ├── app/blueprints/               # REST API 模块
│   ├── app/middleware/auth.py        # 鉴权装饰器
│   ├── app/services/                 # 复杂业务逻辑
│   ├── sql/                          # 数据库迁移
│   ├── run.py                        # 本地入口
│   └── requirements.txt               # Python 依赖
├── docs/                             # 文档、本 PDF 源文件
├── public/                           # 静态资源 (图片)
├── vercel.json                       # Vercel SPA 配置
├── DEPLOY.md                         # 部署步骤
└── package.json                      # 前端 npm 脚本
```

15. 日常开发命令速查

15.1 前端

```
npm install
npm run dev      # http://localhost:8080
npm run build    # 生产构建，检查能否通过
npm run lint
```

15.2 后端

```
cd backend && source .venv/bin/activate
python run.py
# 或
FLASK_APP=app flask run --port 5001
```

15.3 联调 (AI 示例)

```
# 终端 1
cd backend && flask run --port 5001

# 终端 2
VITE_API_URL=http://127.0.0.1:5001/api npm run dev -- --port 8080
```

15.4 Git

```
git status
git add <files>
git commit -m "描述做了什么、为什么"
git push origin main
```

15.5 生成本 PDF

```
cd /Users/yixinzhou/Desktop/EduSync
python3 scripts/md_to_pdf.py \
  docs/EduSync-Full-Stack-Learning-Guide.md \
  docs/EduSync-Full-Stack-Learning-Guide.pdf
```

16. 术语表（中英对照）

中文	English	说明
前端	Frontend	React 跑在浏览器
后端	Backend	Flask 跑在服务器
全栈	Full-stack	前端 + 后端 + 库
接口 / API	API / Endpoint	URL + 方法 + 返回 JSON
部署	Deploy	把代码发布到公网
构建	Build	源码 → 可运行产物
环境变量	Environment variable	配置密钥和 URL
令牌	Token / JWT	登录凭证
蓝图	Blueprint	Flask 路由模块
迁移	Migration	SQL 改表结构
流式响应	Streaming / SSE	AI 逐字输出
单页应用	SPA	一个 index.html + JS 路由

17. 延伸阅读与官方文档

主题	链接
React 官方	https://react.dev
Vite	https://vitejs.dev
Flask	https://flask.palletsprojects.com
Supabase Docs	https://supabase.com/docs
Vercel Docs	https://vercel.com/docs
Railway Docs	https://docs.railway.app
TanStack Query	https://tanstack.com/query
DeepSeek API	https://platform.deepseek.com/api-docs
项目部署	仓库内 <code>DEPLOY.md</code>
AI 架构	仓库内 <code>docs/AI-ARCHITECTURE.md</code>
OAuth 专题 PDF	<code>docs/EduSync-Google-0Auth-Deploy-Learning-Guide.pdf</code>
老师测试指南 PDF	<code>docs/EduSync-Teacher-Guide.pdf</code>

18. 总结：你现在该记住的十句话

1. 用户只访问 Vercel 前端；API 在 Railway；数据在 Supabase。
2. `apiFetch` + JWT 连接前端与 Flask。
3. `VITE_*` 改后要重新 build / redeploy Vercel。
4. `service_role` 永远不能进前端或 GitHub。
5. 新数据库能力 = 先写 SQL 在 Supabase 跑，再写 API，最后写页面。
6. Google 登录是 Supabase Auth + 你自己的 `/auth/callback` 页面。
7. AI 密钥只在 Railway；老师端 Beta 只读工具。
8. React Query 管数据；AuthContext 管登录态。
9. 出问题：F12 Network → Railway Logs → Supabase Logs。
10. Cursor 写代码，你来理解链路——这样才能独立改 bug 和面试讲项目。

